

Wanderwise AI

Multi-Agent Adaptive Trip Planner with Custom MCP Server

A web app that creates personalized trips, recommends hotels/hostels and food based on route and budget, tracks live progress through checkmarks, and suggests itinerary changes through a chatbot with user approval. AI agents use a custom MCP server for tool execution instead of direct tool calling.

Purpose of this document

This brief explains the full project idea for teammates. It covers the user experience, major features, examples, live trip assistant behavior, approval-based changes, weather updates, and the planned folder/file structure for Frontend, Backend, and AI.

Technology direction: Frontend in React.js + Tailwind CSS, Backend in Node.js + Express.js + MongoDB, and AI service in Python + FastAPI with a custom MCP server.

1. Project Overview

Wanderwise AI is not just a normal trip planner. A normal planner gives a static list of places. Wanderwise creates a travel plan that can be used before and during the trip. Before the trip starts, it helps the user plan destination, budget, stay, food, and route. After the trip starts, it becomes a live travel companion.

The core idea is simple: the user enters basic trip details, the system creates a high-level plan, the user can edit it until the trip starts, and once the trip starts the app shows a detailed day-by-day checklist. A chatbot stays visible on the right side so the user can ask for changes anytime.

Example: A user creates a 1-day Delhi trip with a budget of Rs. 4000. Wanderwise suggests staying near Connaught Place/Paharganj, visiting India Gate, Humayun's Tomb, Lodhi Garden, Khan Market, and CP, while also recommending transport, lunch, dinner, and backup options.

Main product promise:

- Plan a trip based on destination, dates, budget, interests, food type, stay type, and travel style.
- Suggest hotels/hostels based on the places planned for each day, not only based on the city name.
- Suggest food places based on quality, food preference, budget, and route distance.
- Give transport guidance between places using cab, metro, bus, auto, or walking where useful.
- Let users check off activities during the trip and ask the assistant what to do next.
- Suggest changes for missed places, weather problems, delays, or user mood changes, but apply changes only after user approval.

High-level user journey:

Login / Open App

- > Upcoming Trips
- > Plan a Trip
- > AI creates high-level overview
- > User edits before trip starts
- > Start Trip
- > Detailed checklist + chatbot
- > User completes, moves, drops, or reschedules activities
- > Trip moves to Completed Trips

2. Main App Flow

The app layout should be simple and practical. The left sidebar has two main options: Upcoming Trips and Completed Trips. In Upcoming Trips, the user can see planned trips and a Plan a Trip button. Each trip card shows destination, dates, budget, trip status, and buttons like View Details or Start Trip.

Before the trip starts, the user sees a high-level overview, not every tiny checkpoint. This is useful because the user may still want to edit budget, food preference, interests, start date, or places to avoid. Once the user clicks Start Trip, the trip becomes active and the app shows the detailed day-wise checklist.

Status	Meaning	Example
Draft	Basic details added, but plan is not generated yet.	User entered Delhi, 1 day, Rs. 4000.
Planned	High-level itinerary is generated and still editable.	Morning monuments, afternoon garden, evening CP.
Started	Trip is active with activity checkboxes and chatbot.	User checks India Gate as completed.
Completed	Trip is finished and visible in Completed Trips.	Final summary shows visited and missed places.

Plan a Trip form fields:

- Destination, start date, end date, total budget, travelers, food preference, stay preference, travel style, and interests.
- The user should also be able to add places to avoid, preferred travel pace, and any special notes.

3. Key Features With Examples

Feature	What It Does	Short Example
Personal Trip Plan	Creates a trip using budget, dates, interests, food, and stay preference.	Plan a 1-day Delhi trip under Rs. 4000.
Day-Wise Itinerary	Breaks the trip into morning, afternoon, evening, and night.	India Gate -> Humayun's Tomb -> Lodhi Garden.
Smart Stay Optimizer	Suggests where to stay based on actual daily route.	Stay near CP/Paharganj for Delhi Day 1.
Hotel/Hostel Suggestions	Shows stay options based on budget, route distance, type, and food nearby.	Budget hostel near Paharganj.
Food Suggestions	Recommends lunch, dinner, cafes, and local food based on preference and route.	Lunch near Khan Market.
Transport Guidance	Suggests cab, metro, bus, auto, or walking with time/cost estimates.	Cab from India Gate to Humayun's Tomb.
Weather Backup	Suggests safer alternatives if rain/heat affects outdoor places.	Move Lodhi Garden and add National Museum.
Chat Assistant	User can ask questions during the trip.	I finished lunch. What next?
Activity Checkmarks	Each activity can be completed, pending, skipped, or moved.	India Gate done, Lodhi Garden pending.
Move/Drop Places	If delayed, app asks whether to move or drop pending places.	Move Khan Market to Day 2 or drop it.
Important behavior: The app should suggest changes, but it should not automatically change the itinerary without user permission.		

4. Detailed Example: 1-Day Delhi Trip Under Rs. 4000

Input: Delhi, 1 day, Rs. 4000, 1 traveler, vegetarian food, budget hotel/hostel, interests in monuments, gardens, local food, and shopping.

Recommended stay:

Stay near Connaught Place, Janpath, or Paharganj. This area is central, budget-friendly, close to New Delhi Railway Station, connected by metro, and convenient for India Gate, Humayun's Tomb, Lodhi Garden, Khan Market, and dinner near CP.

Time	Plan	Details
8:30 AM	Start from hostel near Paharganj	Budget stay around Rs. 900-1300/night.
9:00 AM	Visit India Gate	Auto/cab Rs. 120-200, 15-25 min. Checkbox: completed.
10:30 AM	Go to Humayun's Tomb	Cab/auto Rs. 180-300, 20-30 min. Ticket around Rs. 40.
12:45 PM	Lunch near Khan Market/Nizamuddin	Vegetarian meal or cafe lunch, Rs. 250-500.
2:00 PM	Visit Lodhi Garden	Short auto/cab, Rs. 100-180. Entry free.
4:00 PM	Khan Market or Janpath	Cafe/snacks/shopping, Rs. 150-300.
6:30 PM	Connaught Place evening walk	Metro/cab/auto, Rs. 40-200.
8:00 PM	Dinner at CP	Budget vegetarian dinner, Rs. 300-600.
9:30 PM	Return to stay	Auto/cab Rs. 100-200.

Budget estimate: Stay Rs. 1100, transport Rs. 800, food Rs. 1000, entry Rs. 40, snacks/shopping buffer Rs. 700, emergency buffer Rs. 360.

5. Live Trip Companion and Approval-Based Changes

After the trip starts, the app should feel like a companion. The user can talk to the assistant from the right side chatbot, and the app also knows which activities are completed because of checkmarks.

User Message	Assistant Should Check	Assistant Response
I finished lunch. What should I do now?	Completed activities, current time, pending places, route.	Go to Lodhi Garden now. It is nearby, free, and fits the schedule.
Traffic was bad. I only visited 3 places.	Pending places, time left, next day capacity.	Move Khan Market to Day 2, drop it, or keep only CP dinner?
I do not want India Gate today. Move it to Day 3.	Day 3 plan, route, time, budget impact.	I can move India Gate to Day 3 evening. Should I update?
It is raining heavily.	Outdoor places, weather, indoor alternatives.	Move Lodhi Garden and add National Museum today. Should I update?

Approval rule:

User asks for change

- > AI suggests update
- > Backend saves suggestion as pending
- > Frontend shows popup/buttons
- > User clicks Yes
- > Backend applies change to itinerary

Weather popup example: Heavy rain expected near Lodhi Garden and India Gate today. Suggested update: move Lodhi Garden to Day 2, move India Gate to Day 3, add National Museum today, and keep dinner at Connaught Place. Should I update today's plan?

6. Multi-Agent AI System With Custom MCP Server

The AI part should be in the separate AI folder using Python. The backend calls the Python AI service through HTTP APIs. Inside the AI layer, agents should not directly call external tools. Instead, they communicate with a custom MCP server, and the MCP server exposes all tool actions in a controlled way.

Correct AI architecture: agents do the reasoning and planning; the custom MCP server executes tool operations such as weather checks, hotel/place search, distance calculation, itinerary updates, notification creation, and agent logging.

Frontend

- > Backend API
- > Python AI Service
- > Multi-Agent Workflow
- > Custom MCP Server
- > MCP Tools / external data / structured operations

Agent	Responsibility	Example
Planner Agent	Creates overall plan structure.	Plan Delhi in 1 day under Rs. 4000.
Route Agent	Orders places and reduces travel.	Group India Gate, Humayun's Tomb, Lodhi Garden, CP.
Stay Agent	Uses MCP tools for hotel/hostel options.	Find stays near CP/Paharganj.
Food Agent	Uses MCP tools for food options.	Find vegetarian lunch near Khan Market.
Transport Agent	Uses MCP distance/route tools.	Estimate CP to India Gate travel.
Weather Agent	Requests weather data through MCP.	Replace outdoor garden with indoor museum.
Companion Agent	Chats with user during trip.	I finished lunch, what next?
Critic Agent	Checks if plan is realistic.	Avoid too many far places.
Writer Agent	Creates final clean output.	Readable itinerary.
MCP Tool	Purpose	Example Use
get_weather	Check weather for planned places and dates.	Rain near Lodhi Garden.
search_hotels	Find hotel/hostel suggestions by budget, area, route distance.	Budget hostel near Paharganj.
search_places	Find attractions, indoor alternatives, markets, cafes.	National Museum backup.
calculate_distance	Estimate distance/time between points.	CP to India Gate.
save_itinerary	Store generated itinerary through controlled tool action.	Save high-level plan.
update_itinerary	Apply approved itinerary changes.	Move India Gate after user confirms.
create_notification	Create popup/in-app/email notifications.	Weather alert popup.
store_agent_log	Store what each agent decided and why.	Route Agent avoided far places.

7. Frontend Folder Structure

The frontend should follow the same readable pattern as the Library Management frontend. Keep the structure simple: common visual pieces in components, page-level screens in pages, app shell files in layout, modal/popup screens in popups, and Redux state in store/slices.

```
Frontend/  
  
  src/  
  
    App.jsx  
  
    index.css  
  
    main.jsx  
  
    assets/  
  
      logo.png  
  
      logo-with-title.png  
  
      placeholder.jpg  
  
      map-pin.png  
  
      hotel.png  
  
      hostel.png  
  
      food.png  
  
      weather.png  
  
      transport.png  
  
      checklist.png  
  
      chat.png  
  
      notification.png  
  
    components/  
  
      UpcomingTrips.jsx  
  
      CompletedTrips.jsx  
  
      PlanTrip.jsx  
  
      TripOverview.jsx  
  
      ActiveTrip.jsx  
  
      ActivityChecklist.jsx  
  
      TripChatbot.jsx  
  
      StaySuggestions.jsx  
  
      FoodSuggestions.jsx  
  
      BudgetSummary.jsx  
  
      MapExplorer.jsx  
  
    layout/
```

```
Header.jsx
SideBar.jsx
pages/
  Home.jsx
  Login.jsx
  Register.jsx
  ForgotPassword.jsx
  OTP.jsx
  ResetPassword.jsx
  Dashboard.jsx
  TripDetails.jsx
popups/
  WeatherAlertPopup.jsx
  ConfirmChangePopup.jsx
  TripEditPopup.jsx
  ActivityDetailsPopup.jsx
  StayDetailsPopup.jsx
  FoodDetailsPopup.jsx
  ProfileModal.jsx
  SettingPopup.jsx
store/
  store.js
  slices/
    authSlice.js
    tripSlice.js
    itinerarySlice.js
    activitySlice.js
    chatSlice.js
    suggestionSlice.js
    notificationSlice.js
    mapSlice.js
```

Why this structure is readable:

- components contains the main reusable app sections and trip UI pieces.
- layout contains Header and SideBar, the shell that appears across the app.

- pages contains route-level pages like Login, Register, Dashboard, and TripDetails.
- popups contains modal UI such as weather alert, confirm change, trip edit, and detail popups.
- store/slices keeps Redux logic separated by feature: auth, trips, itinerary, activity, chat, suggestions, notifications, and map.

8. Backend Folder Structure

The backend is the control center. It stores users, trips, itineraries, activities, suggestions, chat history, notifications, and agent logs. It also calls the Python AI service.

```
Backend/  
  server.js  
  app.js  
  config/  
    config.env  
  Database/  
    db.js  
  models/  
    User.js  
    Trip.js  
    Itinerary.js  
    Activity.js  
    StaySuggestion.js  
    FoodSuggestion.js  
    ChatMessage.js  
    ChangeSuggestion.js  
    Notification.js  
    AgentLog.js  
  routes/  
    authRoutes.js  
    tripRoutes.js  
    itineraryRoutes.js  
    activityRoutes.js  
    chatRoutes.js  
    suggestionRoutes.js  
    notificationRoutes.js  
    agentLogRoutes.js  
  controllers/  
    authController.js  
    tripController.js  
    itineraryController.js
```

```
activityController.js
chatController.js
suggestionController.js
notificationController.js
agentLogController.js
services/
  aiService.js
  notificationService.js
  emailService.js
  weatherService.js
middleware/
  authMiddleware.js
  errorMiddleware.js
utils/
  asyncHandler.js
  apiResponse.js
  validators.js
```

9. AI Folder Structure With MCP Server

The AI folder should be a separate Python service, preferably using FastAPI. It also contains the custom MCP server. The Node backend calls the AI service. The AI service runs agent workflows. When agents need real data or actions, they request MCP tools from the MCP server instead of calling tools directly.

```
AI/

  ai_service/

    main.py

    agents/

      planner_agent.py

      route_agent.py

      stay_agent.py

      food_agent.py

      transport_agent.py

      budget_agent.py

      weather_agent.py

      companion_agent.py

      critic_agent.py

      writer_agent.py

    workflows/

      trip_overview_workflow.py

      detailed_itinerary_workflow.py

      chat_workflow.py

      replan_workflow.py

    prompts/

      trip_overview_prompt.txt

      detailed_itinerary_prompt.txt

      replan_prompt.txt

      chat_prompt.txt

  mcp_server/

    server.py

    tools/

      weather_tool.py

      hotel_tool.py

      places_tool.py

      distance_tool.py
```

```
itinerary_tool.py  
notification_tool.py  
agent_log_tool.py
```

Endpoint	Purpose	Used When
POST /ai/trip-overview	Generate high-level trip overview using agents and MCP tools.	After Plan a Trip form.
POST /ai/detailed-itinerary	Generate low-level checklist itinerary.	When user clicks Start Trip.
POST /ai/chat	Answer user message or create proposed itinerary change.	During active trip chatbot use.
POST /ai/replan	Create proposed update for weather, delay, missed place, or user change.	Before user approves a change.

Frontend request

- > Express backend route
- > Controller reads MongoDB trip data
- > aiService.js sends request to Python AI service
- > Python agents reason about the task
- > Agents request tool actions from custom MCP server
- > MCP server returns structured tool results
- > Python AI returns final plan/suggestion
- > Backend saves result in MongoDB
- > Frontend displays updated result

10. Development Roadmap

Phase	What To Build	Result
1. Backend foundation	server.js, app.js, db.js, User model, Trip model.	Server runs and MongoDB connects.
2. Basic trips	Create trip, upcoming trips, completed trips, edit trip.	User can save planned trips.
3. Mock itinerary	Return sample overview and detailed checklist without AI.	Frontend can be built without waiting for Python AI.
4. Activity tracking	Complete, uncomplete, skip, move activity.	Live checklist works.
5. Chat suggestion flow	Chat message creates pending ChangeSuggestion.	AI-style changes require approval.
6. Python AI service	FastAPI endpoints and agent workflows.	Agents can generate trip plans.
7. Custom MCP server	Expose weather, hotel, place, distance, itinerary, notification, and log tools.	Agents use MCP tools instead of direct tool calling.
8. Frontend structure	Build src with assets, components, layout, pages, popups, store/slices.	Readable UI code organization.
9. Weather alerts	Weather check creates popup suggestion.	User can update plan due to weather.
10. Map feature	Show hotels, places, food, routes, and photos.	Visual trip planner experience.
Final summary: Wanderwise AI is an adaptive trip planner with route-aware stay suggestions, food and transport guidance, live checkmarks, chatbot-based itinerary editing, weather suggestions, approval-based changes, and a custom MCP server that handles tool execution for AI agents.		